

AI 编程进阶与 Agent 时代

项目级 AI 技巧 · 代码质量 · 软件工程的未来

"世界上第一个 AI 软件工程师"——它真的能替代程序员吗？

南京大学 · 《软件工程与计算 II》 · 2026 春季学期

模块	核心问题
项目级 AI 编程技巧	如何让 AI 真正理解你的项目
AI 辅助代码质量	Debug · 测试 · 审查 · 重构
Agent 时代的软件工程	人与 AI 的关系将走向何方

开场案例：Devin 的诞生与争议（2024）

"Devin 是世界上第一个 AI 软件工程师。" ——Cognition AI, 2024-03-12

时间线	事件
2024-03-12	Cognition AI 发布 Devin，视频演示令全球震惊
宣传数据	SWE-bench 基准测试：独立解决 13.86% 的 GitHub Issue
全球反应	"程序员要失业了？"——Twitter/X 上数千条讨论
独立复测	多个团队实测后：实际表现 1%—3% ，远低于宣传
2024 年底	各大公司陆续发布自己的 AI 编程 Agent：Claude Code · GitHub Copilot Workspace · Cursor Agent

根因分析：我们真正应该问的问题

真正的问题不是"AI 能不能替代程序员"，而是"人与 AI 如何高效协作"

人与 AI 的四种关系（从历史到未来的演进）：

① 人 独 立	工程师独自解决所有问题	
② AI 辅 助 人	AI 工具提高工程师生产效率	← 我们现在所处的阶段
③ 人 辅 助 AI	工程师为 AI Agent 提供方向和审查	← 正在进入
④ AI 独 立	AI 自主完成端到端软件工程任务	← 未来（距今还很远）

今天的定论：处于②→③过渡期。

最高效的工程师 = 能精确指导 AI、能审查 AI 输出、能处理 AI 无法处理的边界

💡 Devin 的案例告诉我们：AI 的宣传往往超前于现实；但趋势的方向是真实的

本讲路线图

模块	核心问题	时长
PART 1	如何在真实项目中高效使用 AI?	~20 min
PART 2	AI 如何帮助提升代码质量?	~15 min
PART 3	Agent 时代的软件工程新思考	~15 min

这一讲是对前两讲的深化——工具会用了，如何用得更专业？

PART 1

项目级 AI 编程技巧

真实场景 · 10 个高效提示词 · 跨文件理解

提示词进阶：精准控制 AI 的边界

AI 在项目中最常见的失误：**改动范围失控**（你让它修一个 bug，它重写了整个文件）

5 个控制提示词（来自真实使用经验）：

1. "仅修复此问题，不要修改其他代码"
→ 防止 AI 顺手"优化"你不想动的部分
2. "请使用最小修改来解决此问题"
→ 保持代码风格一致，减少 git diff 噪音
3. "换个思路来解决此问题"
→ 当前方案性能差或逻辑混乱，让 AI 从头重想
4. "仔细阅读全文，检查是否还有其他同类问题"
→ AI 容易只看局部；这个提示词强制它做全局审查
5. "请为我梳理当前代码的整体逻辑"
→ 接手陌生项目时，让 AI 帮你生成"项目导览"

提示词进阶：代码质量与可维护性

5 个质量提升提示词：

6. "请提供详细的解释和行内代码注释"
→ 生成可维护的代码，而不是神秘的黑盒
7. "生成测试用例来验证此更改，
包含正常路径、边界条件和异常路径"
→ 比 "生成测试" 更精准，覆盖率更高
8. "增加详细的日志输出，方便追溯错误"
→ Debug 时事半功倍
9. "请简化代码逻辑，去除冗余部分，
但保持功能完全一致"
→ 加 "保持功能一致" 是关键约束，防止 AI 改变行为
10. "优化代码结构，使其更加模块化，
每个函数只做一件事"
→ 明确重构目标（单一职责原则）

跨文件理解：处理真实项目

场景：拿到一个 Java Spring Boot 项目，要新增一个功能

✗ 错误做法（只看当前文件）：

打开 UserController.java, 问 AI "帮我加一个查询用户列表的接口"

→ AI 不知道 UserService 和 UserMapper 已有什么方法

→ 生成重复代码，或与现有逻辑冲突

✓ 正确做法（Cursor Ctrl+Enter，扫描整个项目）：

"这个项目已有哪些用户相关的接口？"

UserMapper 已实现了哪些查询方法？

我要新增 '按角色查询用户列表' 接口，应该在哪些文件里做什么修改？"

→ AI 给出跨文件的完整修改方案

原则：给 AI 的上下文越完整，它的输出越准确——垃圾输入，垃圾输出

实践案例：用 AI 开发飞机大战游戏

真实课程项目演示——从零到可玩游戏，全程 AI 辅助

提示词 1：需求描述

"用 Python + pygame 实现一个简单的飞机大战游戏：

- 玩家飞机可以用方向键移动
- 按空格键发射子弹
- 随机生成敌机，从上方飞来
- 子弹击中敌机得分，敌机碰到玩家游戏结束

先只生成主框架，不要实现所有功能"

提示词 2：迭代添加功能

"现在给敌机加上 AI 自动瞄准行为：

敌机会缓慢向玩家当前位置移动

仅修改敌机的移动逻辑，不要修改其他代码"

关键技巧：分步骤、小批量地让 AI 添加功能，每步验证，不要一次让 AI 实现所有功能

动手实践 (5 分钟)

任务：用 AI 的 Debug 能力修复一段有 Bug 的代码

```
# 这段代码有 3 个 Bug, 让 AI 帮你找出来
def calculate_average(numbers):
    total = 0
    for num in numbers:
        total += num
    return total / len(numbers)    # Bug 1: 空列表时会报错

def find_max(lst):
    max_val = lst[0]               # Bug 2: 空列表时会报错
    for item in lst:
        if item > max_val:
            max_val = item
    return max_val                 # Bug 3: 拼写错误

result = calculate_average([1, 2, 3, 4, 5])
print("平均值:", result)
```

要求：用提示词 4 ("仔细阅读全文，检查是否还有其他同类问题") 让 AI 找出全部 Bug，不要直接告诉 AI Bug 在哪里

PART 2

AI 辅助代码质量

Debug · 测试生成 · 代码审查 · 重构

AI 辅助 Debug：系统化流程

第一步：提供完整上下文

"以下代码在运行时报 `NullPointerException`,
错误堆栈是：[粘贴完整的 `stack trace`]
相关代码是：[粘贴出错方法 + 调用方法]"

第二步：让 AI 分析根因（不要直接让它改代码）

"请分析这个 `NPE` 的根本原因，不要修改代码，先给出分析"

第三步：确认理解后，再让 AI 修复

"你的分析正确，现在用最小修改修复这个问题，
只修改 `UserService.getUserById()` 方法"

第四步：验证修复

"为这个修复生成 2 个测试用例：
一个正常路径（用户存在），一个异常路径（用户不存在）"

为什么分步骤：直接让 AI "修复 Bug" 往往改动范围太大，且你不理解原因

AI 生成测试用例

场景：为 MyBatis Mapper 接口生成测试用例

```
// 提示词: "为以下 Mapper 方法生成 JUnit 5 测试,  
// 使用 @MockitoBean 模拟数据库,  
// 每个方法至少包含: 正常路径、空结果、异常路径三个测试用例"
```

```
@Mapper  
public interface UserMapper {  
    User findByUsername(String username);  
    int insert(User user);  
}
```

AI 生成的测试框架：

```
@ExtendWith(MockitoExtension.class)  
class UserMapperTest {  
    @Mock private UserMapper userMapper;  
  
    @Test void findByUsername_normalPath_returnsUser() { ... }  
    @Test void findByUsername_userNotExist_returnsNull() { ... }  
    @Test void insert_validUser_returnsOne() { ... }  
}
```

AI 辅助代码审查

场景：收到同学的 PR，用 AI 快速找出问题

提示词模板：

"请审查以下代码，重点检查：

1. 潜在的安全问题（SQL 注入、XSS、权限验证缺失）
2. 性能问题（N+1 查询、不必要的循环）
3. 可维护性问题（方法过长、命名不清晰）
4. 边界条件处理（空值、空列表、越界）

给出每个问题的：位置、严重程度（高/中/低）、修改建议
不要重写代码，只给出文字分析"

💡 AI 的代码审查是人工审查的补充，不是替代——安全问题和业务逻辑问题仍需人工判断

停下来想一想

"AI 能完全替代 Code Review 吗？"

能力维度	AI 表现	人工优势
语法和拼写错误	✓ 非常擅长	—
常见安全漏洞 (SQLi、XSS)	✓ 较好	—
性能问题识别	⚠ 局部可以	需要理解整体架构
业务逻辑正确性	✗ 不了解业务	必须由人判断
隐式约束 (如安全合规)	✗ 不了解公司规范	必须由人判断
需求合理性	✗ 只看代码	必须联系产品需求

结论： AI Code Review = 自动化 Linter 的升级版——捕获语法/常见漏洞非常高效，但业务正确性和架构决策必须人工审查

PART 3

| Agent 时代的软件工程

新需求 · 降低复杂度 · 人与 AI 如何协同

什么是软件工程 Agent?

Agent = 能自主执行多步任务的 AI 系统

传统 AI 工具 (被动):

你输入 → AI 输出 → 你执行
(每一步都需要人来触发)

AI Agent (主动):

你描述目标 → Agent 自主规划步骤 → Agent 调用工具执行
→ Agent 检查结果 → Agent 调整计划 → 直到完成目标
(人只需提供目标和最终审查)

当前真实能力 (2025 年实测):

- 修复简单 Bug: Agent 已相当可靠
- 添加单一功能: 在上下文清晰时表现较好
- 端到端开发完整功能: 仍需大量人工指导

让 Agent 理解需求：Spec（规格说明）

最大挑战：Agent 执行的是你描述的需求，不是你脑子里真实的需求

✗ 模糊的任务描述（导致 Agent 跑偏）：

"帮我实现用户注册功能"

✓ 清晰的 Spec（让 Agent 准确执行）：

"实现用户注册功能，具体要求：

1. POST /api/auth/register 接口
2. 请求体: { username, password, email }
3. 密码用 BCrypt 加密后存入 users 表
4. username 和 email 不允许重复，重复时返回 409 状态码
5. 成功返回 { userId, username, message: '注册成功' }
6. 只修改 AuthController.java 和 UserMapper.java
7. 复用已有的 UserService, 不要创建新类"

核心：Spec 越精确，Agent 越可靠——这与需求工程的原则完全一致

降低复杂度：Skill（技能模块化）

问题：复杂项目中，Agent 在多个任务之间切换时容易"忘记上下文"

解决方案：将可复用的任务封装成 **Skill**（技能模块）

项目初始化 Skill（一次定义，反复使用）：

"当我说 '初始化新 API 接口' 时，总是：

1. 在 controller/ 创建 Controller 类（遵循项目的命名规范）
2. 在 service/ 创建对应 Service 接口和实现类
3. 在 mapper/ 创建 Mapper 接口
4. 在 mapper/xml/ 创建 XML 文件
5. 在 entity/ 创建实体类
6. 生成一个基础的 GET 接口作为入口"

代码审查 Skill：

"当我说 '审查这段代码' 时，总是检查：

SQL 注入风险 · NPE 风险 · 事务边界 · 命名规范"

软件工程的未来演进

阶段	特征	工程师角色
① 个人开发时代	一人完成所有工作	全栈工匠
② 团队协作时代	分工、版本控制、代码审查	专业分工
③ 团队 + AI 辅助	AI 加速个人，人做最终决策	高效协作者
④ AI + 人辅助	Agent 主导，人提供 Spec 和审查 ← 正在发生	
⑤ AI 自主时代	AI 完成端到端工程任务	架构师 / 决策者

对你意味着什么？

- 阶段④的核心技能：写清晰的 Spec、审查 Agent 输出、处理边界案例
- 这恰好就是需求工程 + 代码审查 + 测试的综合能力
- 软件工程的基本功，在 AI 时代变得更加重要，而不是更不重要

未来的人机协作模式

人找服务 → 服务找人

传统模式（人找服务）：

用户打开 App → 搜索功能 → 手动操作 → 完成任务

Agent 模式（服务找人）：

Agent 主动感知用户需求

→ 调用多个服务完成任务

→ 只在关键决策点询问用户

→ 完成后汇报结果

新兴交互范式：

- **A2UI (AI to UI)**：AI 根据任务动态生成界面 (CopilotKit)
- **Cowork (协同工作)**：人和 AI 在同一个工作流中实时协作，而不是交替轮流
- **群聊式 Agent**：多个 AI Agent 协作（类似企业微信群），分别承担设计/开发/测试角色

💡 这些不是遥远的未来——部分已经在工具中实现

课堂总结：AI 时代工程师的核心竞争力

变得更重要的能力

- ✓ 清晰表达需求 (Spec 能力)
- ✓ 审查和评估代码质量
- ✓ 系统架构设计能力
- ✓ 测试与边界条件思维
- ✓ 理解业务与用户需求
- ✓ 处理 AI 无法处理的模糊性

变得不那么重要的能力

- ↓ 记忆 API 细节 (AI 能查)
- ↓ 手写样板代码 (AI 能生成)
- ↓ 重复性的格式化工作
- ↓ 单纯的代码输入速度

Devin 的真正启示：AI 最终会变得更强大，但在那之前，能精确指导 AI 的工程师将比以往任何时候都更有价值——因为他们放大了 AI 的能力

作业（二选一）

选题 A：AI 辅助项目实践

选择一个实际项目（课程项目 / 个人项目）：

1. 用 AI 完成至少 **1 个完整功能**（从需求描述到代码实现到测试）
2. 记录完整的提示词迭代过程（不少于 5 轮）
3. 用 AI 生成测试用例并验证功能正确性
4. 提交：代码仓库链接 + 提示词迭代记录 + 总结（哪些步骤 AI 做得好/差）

选题 B：AI 时代工程师角色思考

结合三讲内容，写一篇思考报告（800—1000 字）：

- 在你最感兴趣的技术方向（前端/后端/算法/数据...），AI 改变了哪些工作方式？
- 你认为 5 年后，工程师最需要哪 3 种不可被 AI 替代的能力？
- 你打算如何系统地使用 AI 工具来提升自己的学习和开发效率？

三分钟回顾

问题	答案
Devin 的实际表现与宣传相差多少?	宣传 13.86%，独立复测约 1-3%
人与 AI 的四种关系?	人独立 · AI 辅人 · 人辅 AI · AI 独立
让 AI 生成测试用例时，提示词要包含什么?	正常路径 + 边界条件 + 异常路径
AI Code Review 最大的局限是什么?	不了解业务逻辑和公司规范，需人工判断
Spec 对 Agent 的作用?	Spec 越精确，Agent 执行越可靠
AI 时代变得更重要的能力?	需求表达、代码审查、架构设计、边界思维

下节预告

第 6 讲：前端开发基础

2004 年愚人节，Gmail 的 1 GB 存储让世界震惊——但真正的秘密，是**页面不刷新**就能加载数据的那项技术.....

三讲总结：AI4Coding 的完整图景

08-01 AI 辅助编程入门

└─ 工具生态 · 六大能力 · 提示词四要素

08-02 开发工具链与 Git 工作流

└─ DevOps · Git 四区域 · Vercel 自动部署

08-03 AI 编程进阶与 Agent 时代

└─ 项目级技巧 · 代码质量 · 软件工程的未来演进

核心洞察：AI 工具改变的是效率和交互方式，
不变的是：**清晰的思维、严格的工程纪律、对业务的深刻理解**

南京大学 · 《软件工程与计算 II》 · 2026 春季学期
第 8-03 讲 AI 编程进阶与 Agent 时代

本讲核心收获

- 10 个高效提示词：精准控制 AI 的边界与质量
- AI 辅助代码质量：Debug → 测试 → 审查 → 重构
- Agent 时代：Spec + Skill + Cowork, 软件工程演进路径

工具终将进化，工程思维永远有价值